PROGRAMMING FOR PROBLEM SOLVING

26CSE1002J

PREMANAND S

Assistant Professor

Department of Computational Intelligence

SRM Institute of Science and Technology

Kattankulathur Campus

First Year Engineering

Why Programming?

We solve problems every day.

But what happens when the problem becomes too large, too repetitive, or too complex for a human to handle?

The Big Question

Can we teach a computer how to solve the problem?

That's where Programming begins.

What is Programming?

Programming is the process of telling a computer how to solve a problem.

PROBLEM $\longrightarrow$ **INSTRUCTIONS** $\longrightarrow$ **COMPUTER** $\longrightarrow$ **RESULT**

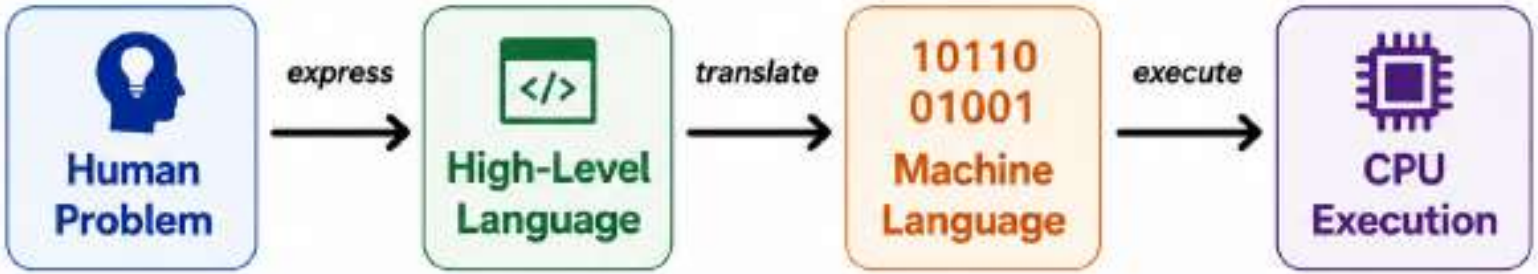
Think Like a Programmer

Understand the problem $\rightarrow$ **Break it into steps** $\rightarrow$ **Express the steps** $\rightarrow$ **Execute and test**

A program is a precise set of instructions that a computer can execute.

How Does a Computer Understand Our Instructions?

Humans think in ideas. Computers execute instructions.



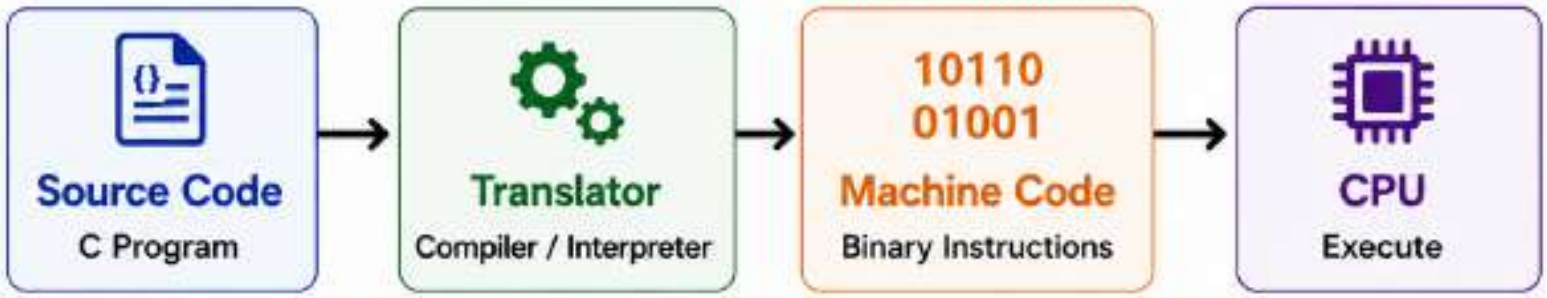
Three Levels of Programming Languages

- 101 010 • **Machine Language** -- Binary instructions - CPU
- MOV ADD • **Assembly Language** -- Mnemonics
- C / C++ Python • **High-Level Language** -- Human-readable languages

Key idea: A program written by humans must eventually become instructions that the computer can execute.

How Is a Program Translated?

We write programs for humans to understand.
The CPU needs machine instructions.



Compiler

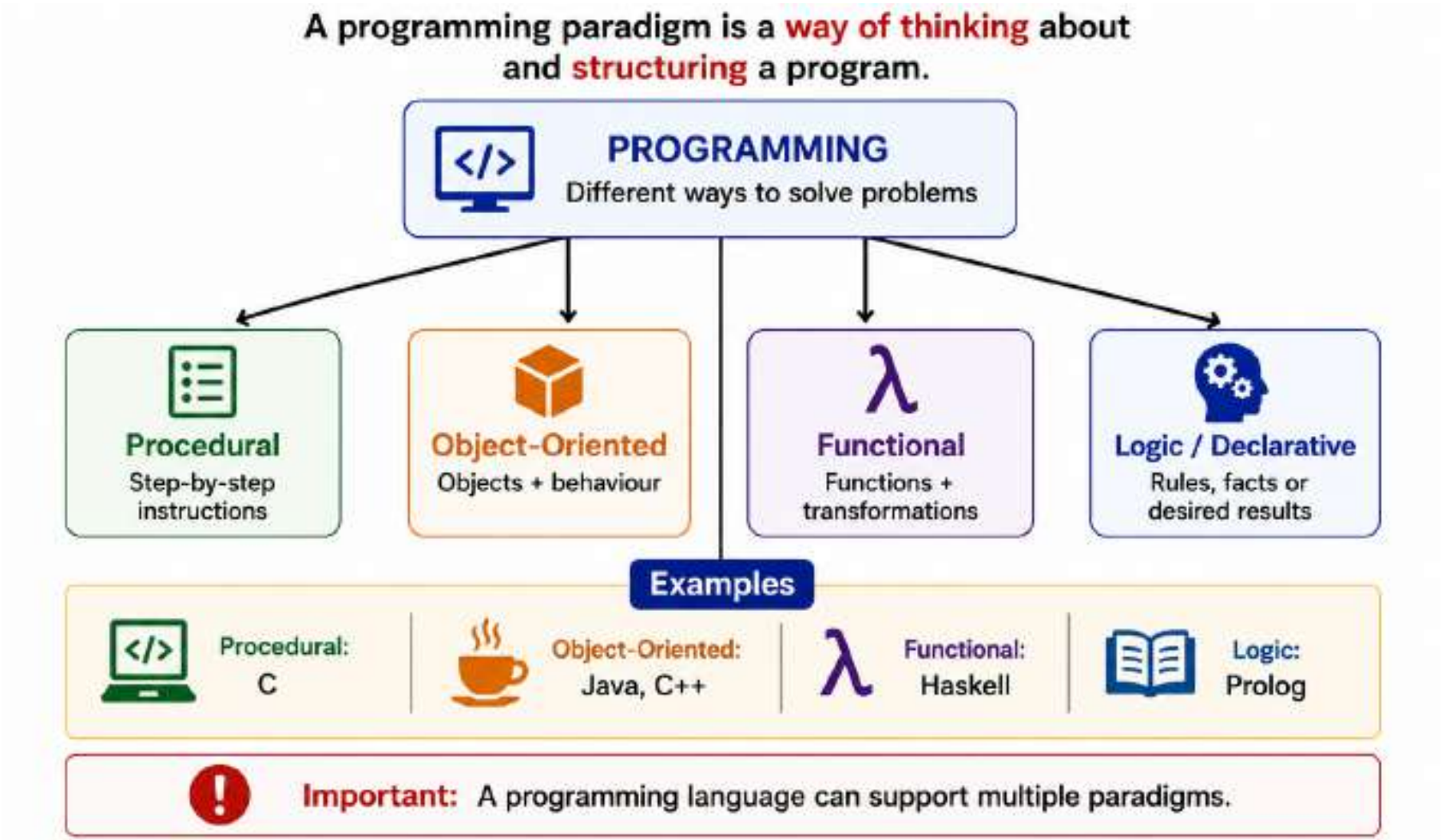
- Translates the program **before** execution
- Produces executable/object code
- C is typically **compiled**

Interpreter

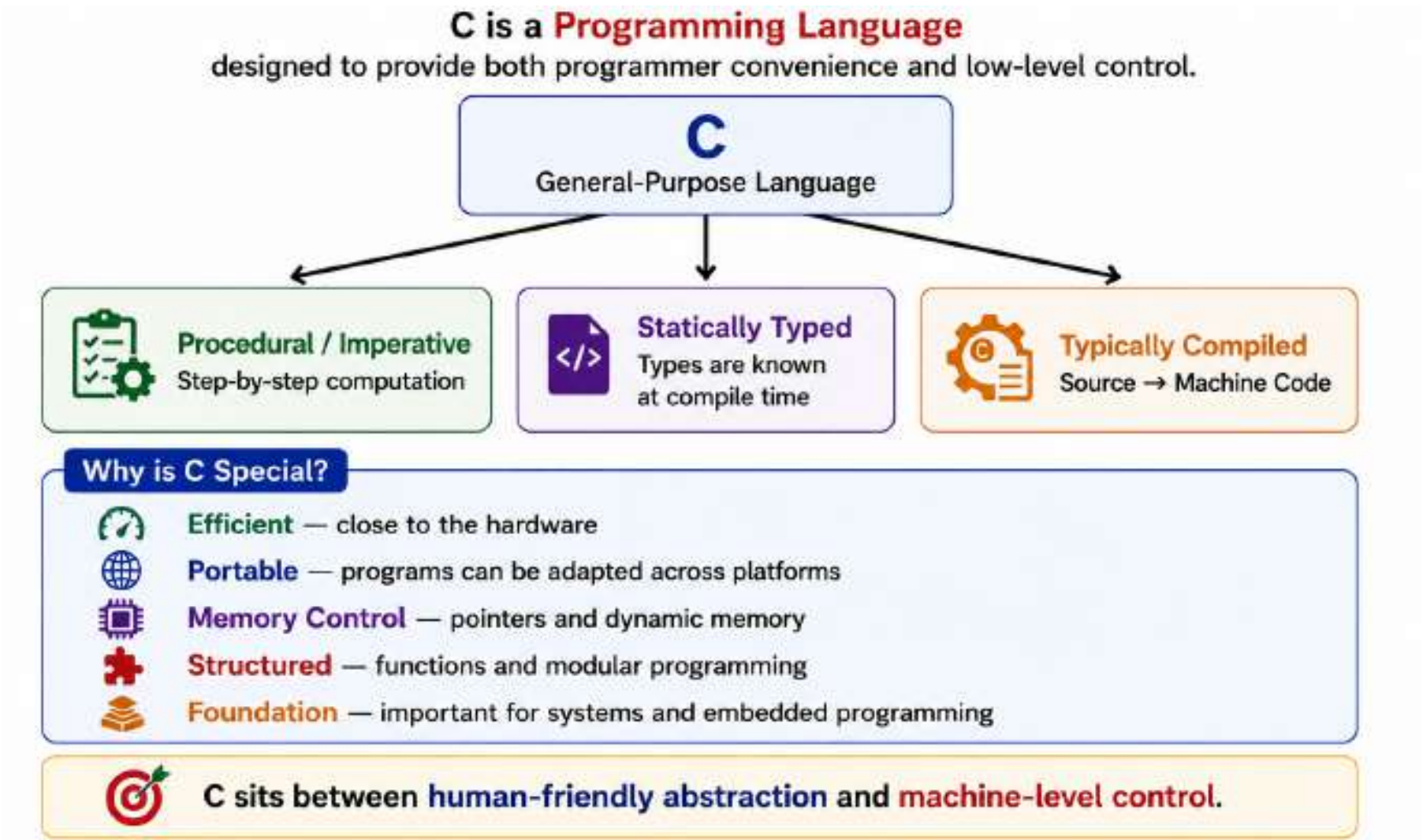
- Executes through a language **implementation**
- Translation and execution are closely **combined**
- Commonly associated with **Python**

Important: Compiler and interpreter describe *how programs are translated/executed* —they are **not programming paradigms.**

Programming Paradigms

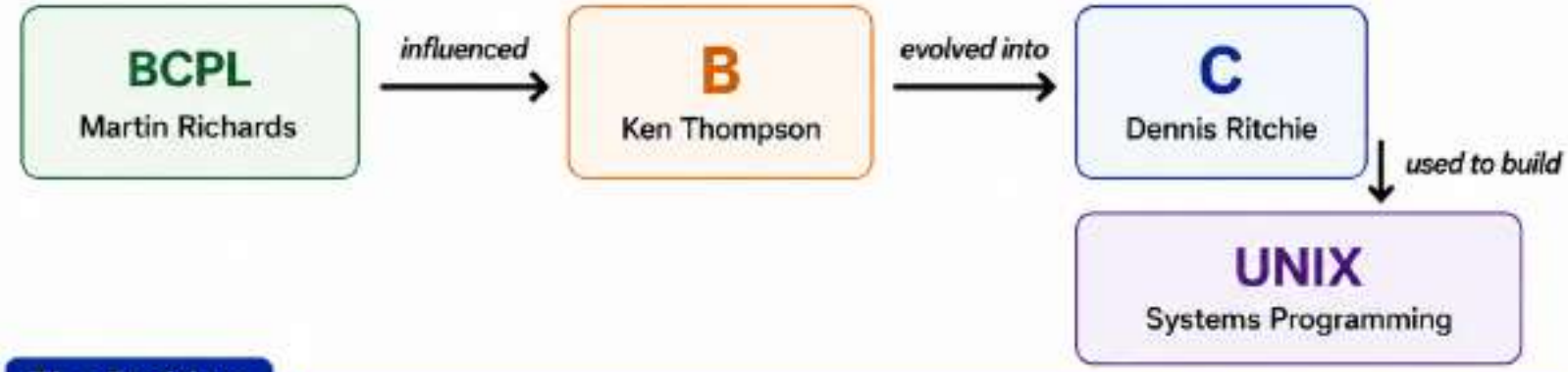


Where Does C Fit?



Why Was C Created?

C was born from a real problem:
How can we build powerful system software efficiently?



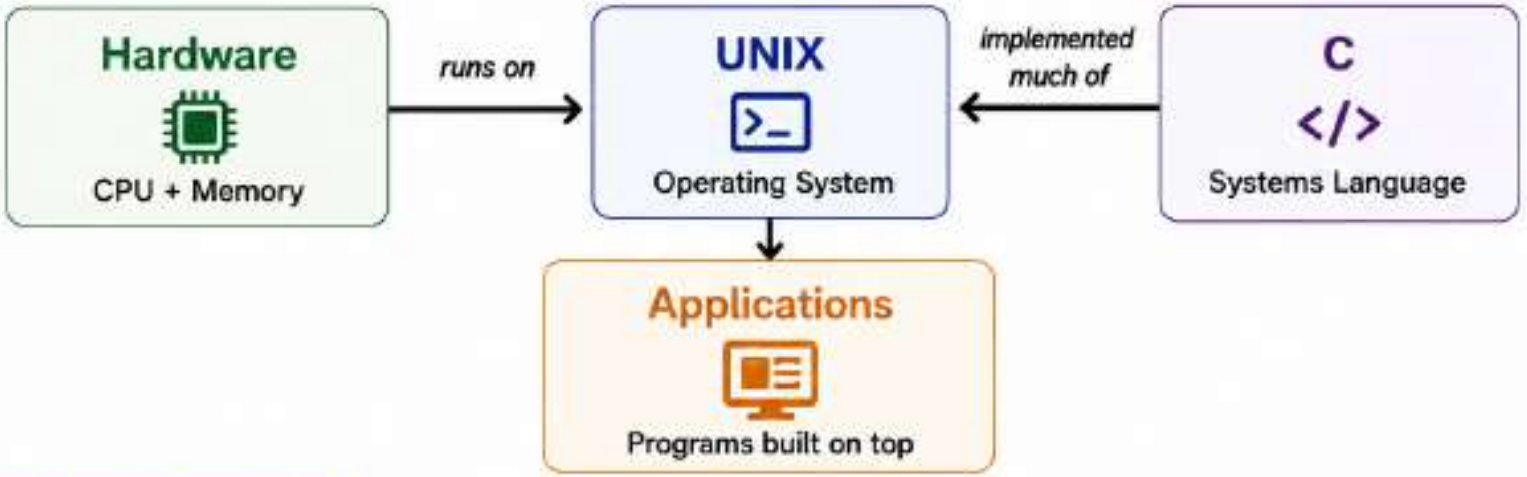
The Problem

- Assembly language offered **low-level control** but was difficult to develop and maintain.
- Higher-level languages offered **abstraction** but were not ideal for some systems programming tasks.
- C aimed to provide a practical balance between **abstraction, efficiency** and **hardware control.**

C was developed at **Bell Labs** during the **early 1970s.**

C and UNIX: A Turning Point in Computing

C became powerful because it was used to build real system software.

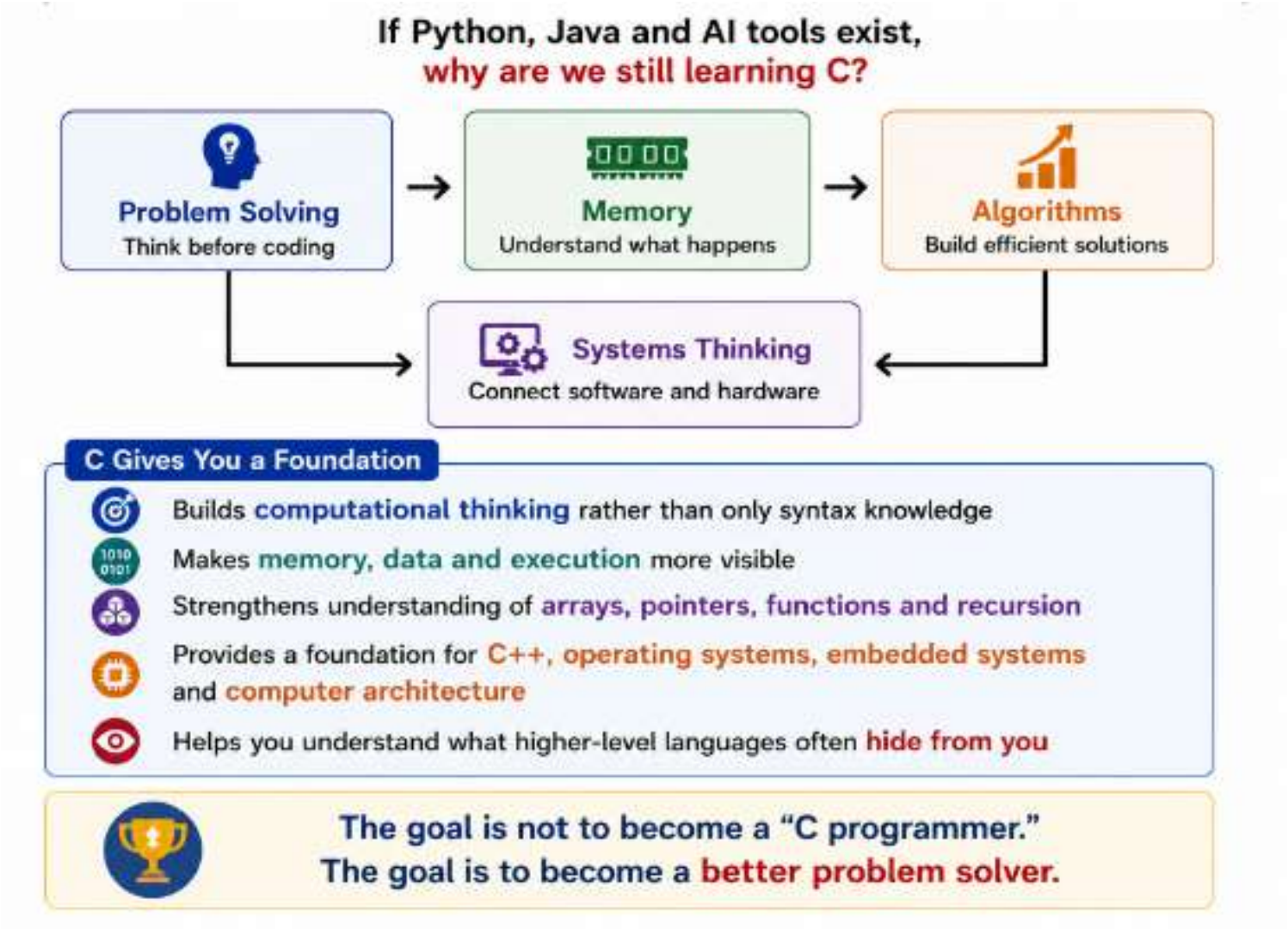


Why Did This Matter?

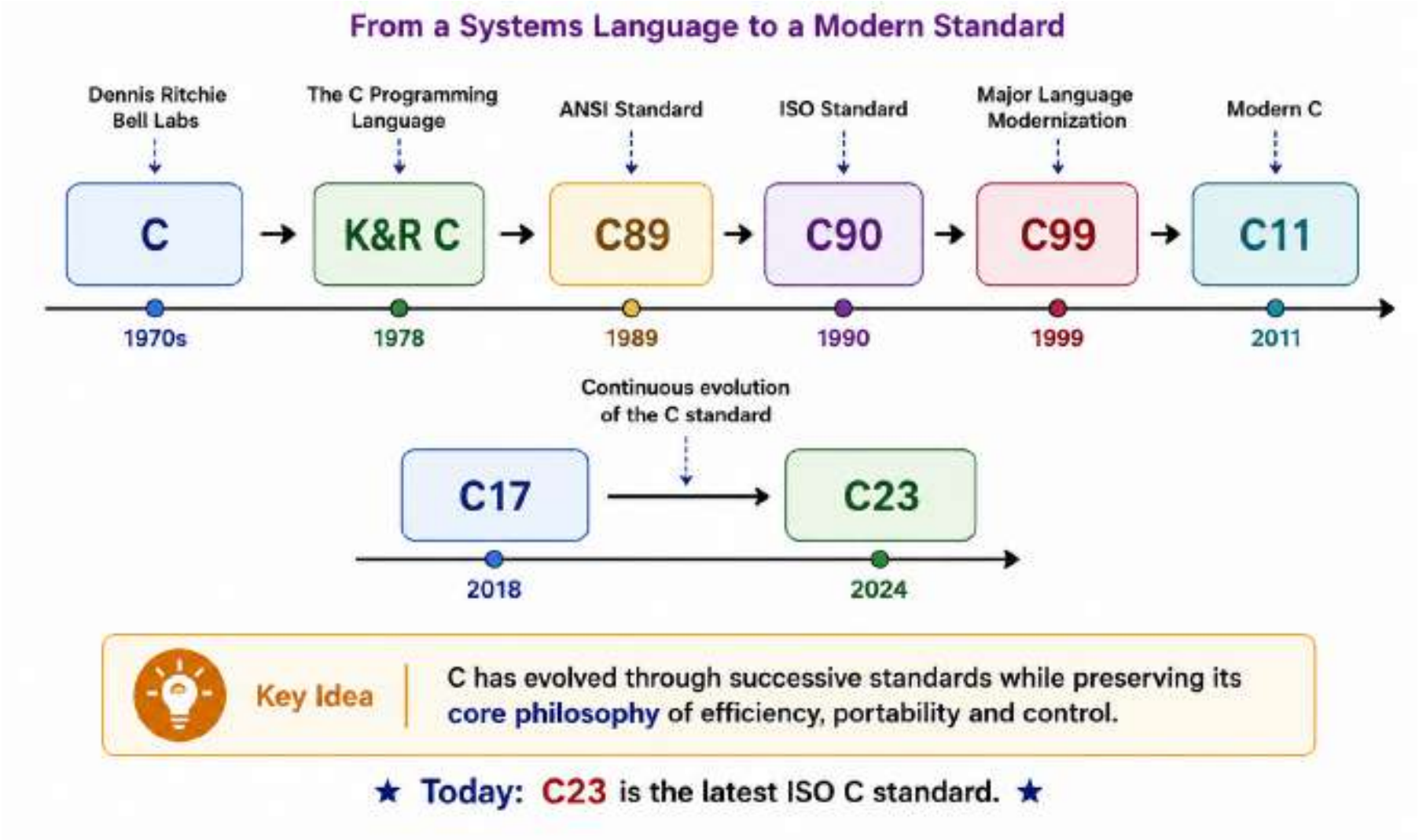
- C allowed large portions of system software to be written more conveniently than assembly.
- Programs written in C could be adapted to different computer architectures.
- The success and portability of UNIX helped C spread beyond its original environment.

C was no longer just a language experiment — it became a practical tool for building systems.

Why learn C in 2026?



Evolution of C?



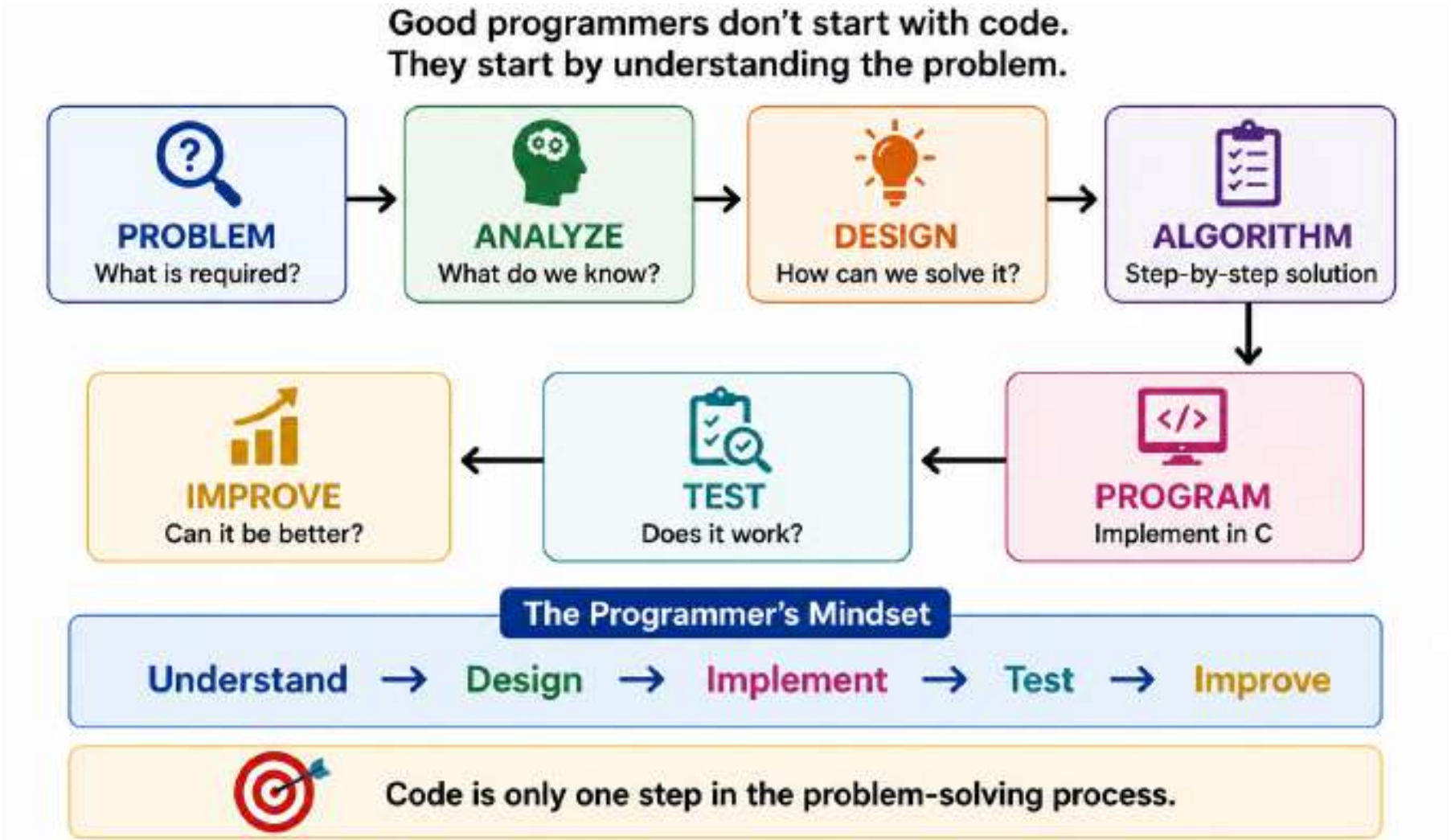
What changed in C?

Standard	Year	Key Highlights
C89	1989	First major ANSI standardization of C
C90	1990	ISO standardization of C
C99	1999	// comments, declarations in `for` loops, variable-length arrays, `inline`, improved integer types
C11	2011	Multithreading support, atomics, `_Generic`, improved Unicode support and safer library features
C17	2018	Clarifications, corrections and defect fixes; largely a refinement of C11
C23	2024	Further language modernization, including `nullptr`, `true/false` as keywords, digit separators and other improvements

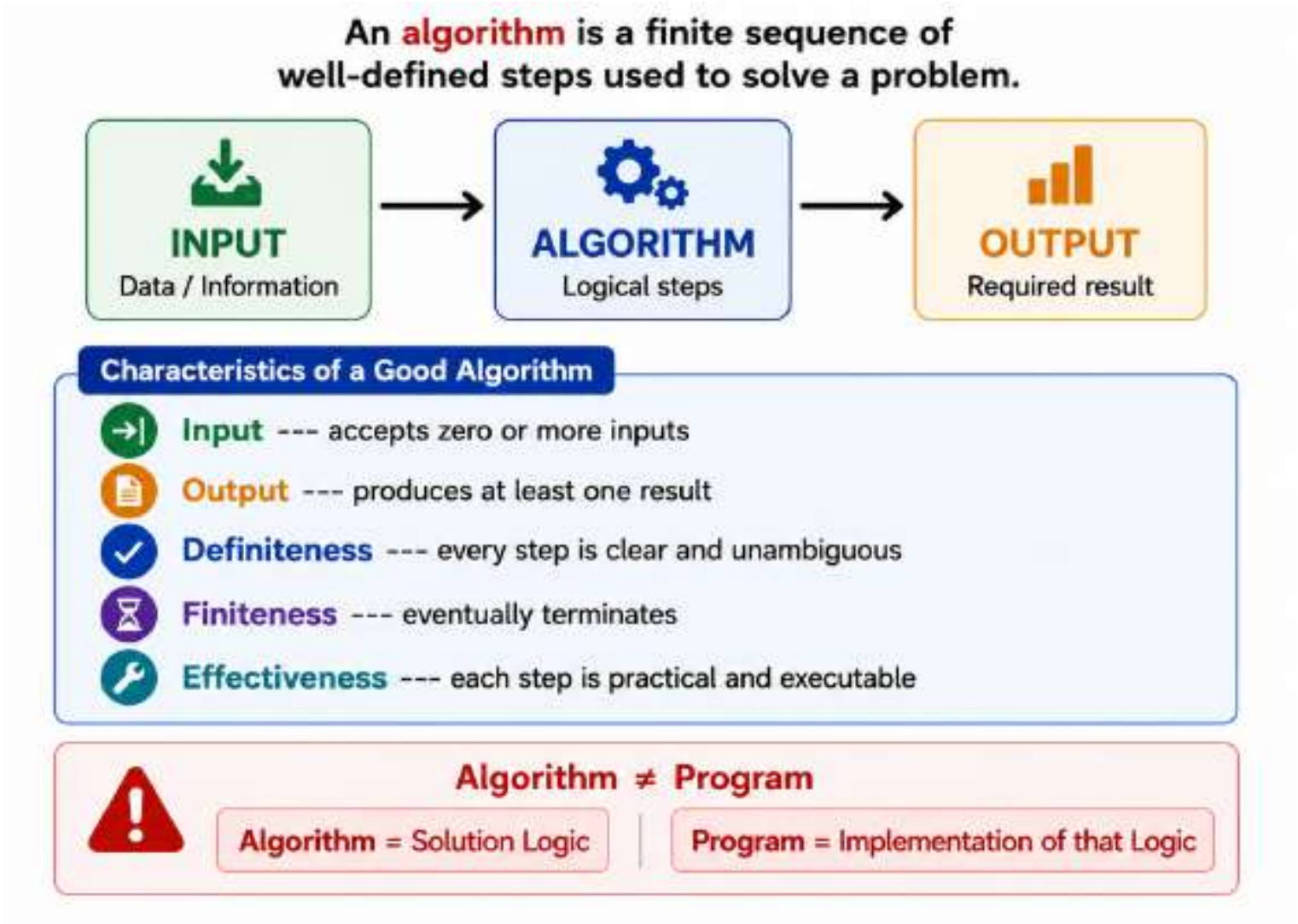
The Important Point

New standards add capabilities and improve the language, but C continues to emphasize efficiency, portability and control.

Before We Write Code: How Do We Solve a Problem¿



What is an Algorithm?



Algorithm Example: Finding the Largest of Three Numbers

Problem: Find the largest among three numbers

INPUT	PROCESS	OUTPUT
A, *B*, *C*	Compare the three values and identify the largest	*Largest*

Algorithm

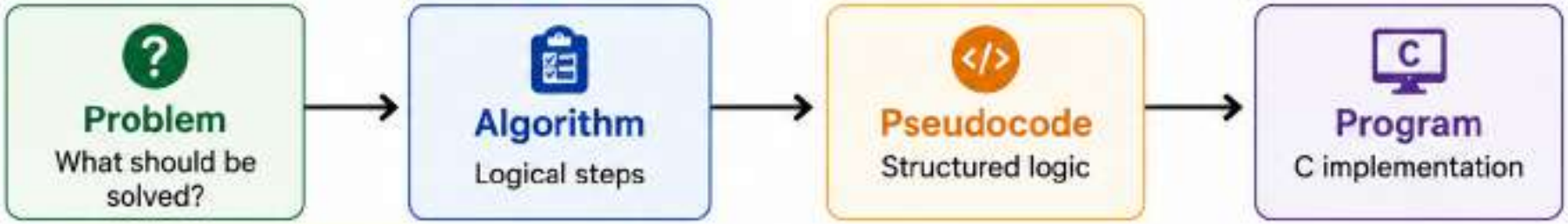
1. Read three numbers *A*, *B* and *C*.
2. Compare *A* with *B* and *C*.
3. If *A* is greater than both, set *Largest* = *A*.
4. Otherwise, compare *B* and *C*.
5. If *B* is greater than *C*, set *Largest* = *B*.
6. Otherwise, set *Largest* = *C*.
7. Display *Largest*.

Think Before You Code

We have solved the problem logically --- without writing C code.

What is Pseudocode?

Pseudocode is a structured way of describing an algorithm using simple, programming-like language.

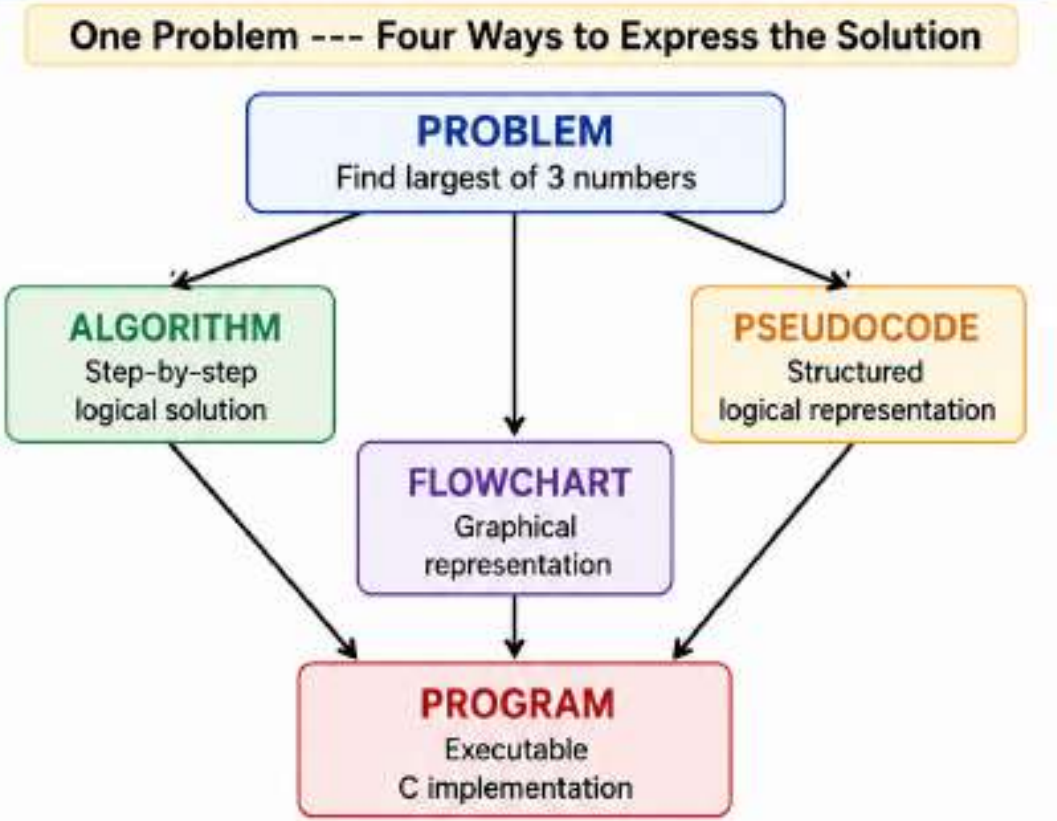


Example: Largest of Three Numbers

```
START
READ A, B, C

IF A > B AND A > C
   Largest <- A
ELSE IF B > C
   Largest <- B
ELSE
   Largest <- C

PRINT Largest
STOP
```

Pseudocode focuses on logic, not programming-language syntax.

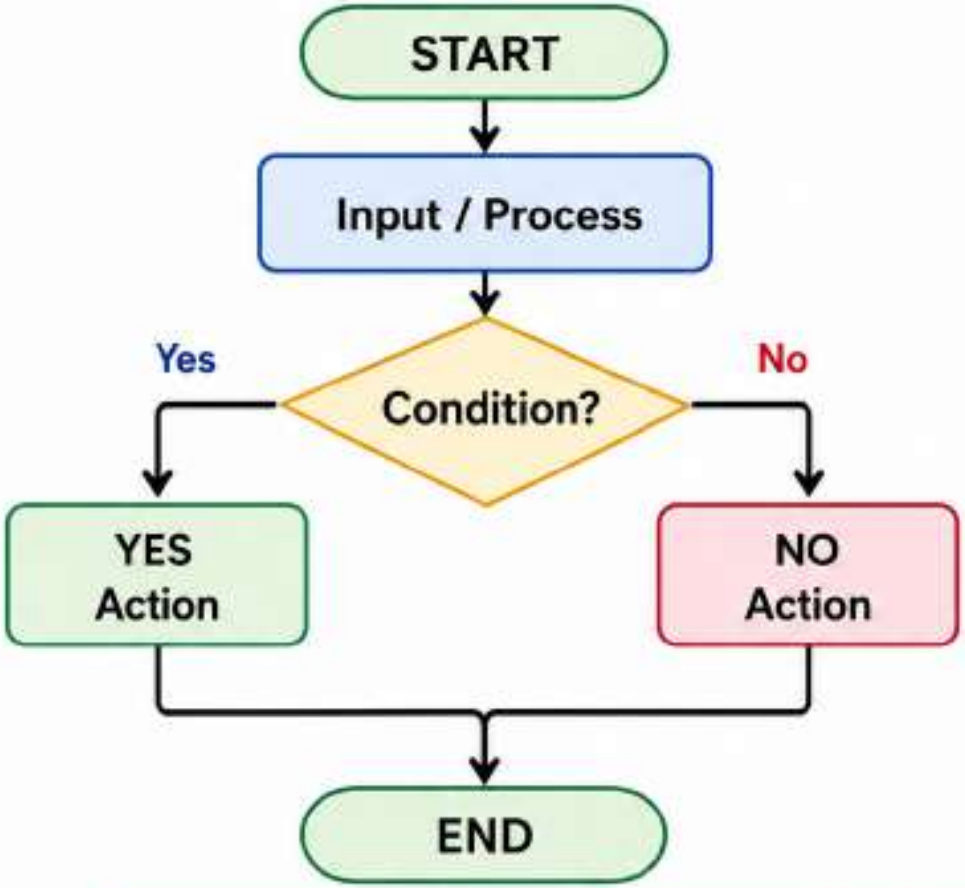
Algorithm Vs Pseudocode Vs Flowchart Vs Program



Representation	Focus	Language Dependent?
Algorithm	Logic / Steps	No
Pseudocode	Structured Logic	No
Flowchart	Visual Logic	No
C Program	Implementation	Yes

What is Flowchart?

A flowchart is a graphical representation of an algorithm.



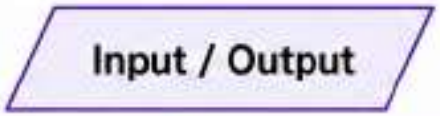
Why Use Flowcharts?

- **Visualize** the flow of a solution
- Make **decisions and branches** easy to see
- Help **identify missing or incorrect logic**
- **Communicate** a solution before writing code

Flowchart = Visual representation of the algorithm

Flowchart Symbol

Each symbol represents a specific type of operation.

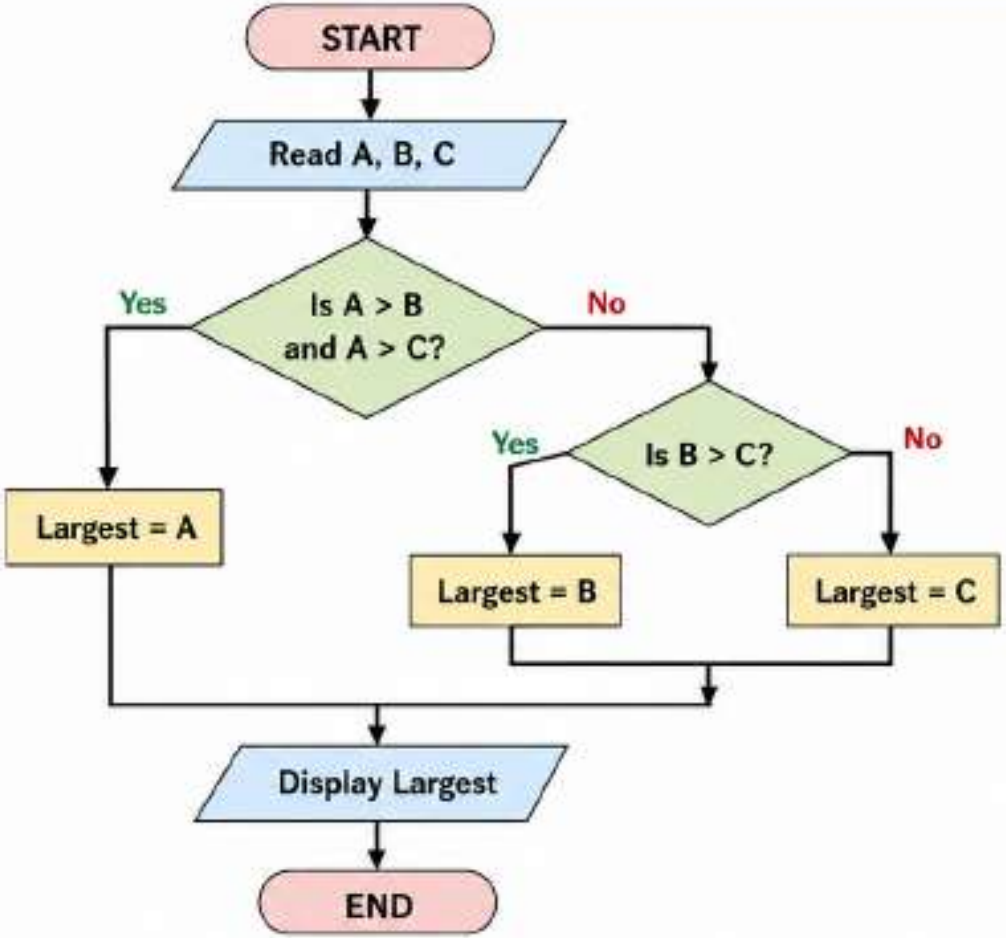
Symbol	Name	Purpose
	Terminal	Indicates the beginning or end of a flowchart
	Process	Represents a calculation or processing step
	Input / Output	Represents reading input or displaying output
	Decision	Represents a condition with alternative paths
	Flow Line	Shows the direction in which the algorithm proceeds

Remember

Shape gives meaning. Arrow gives direction.

Flowchart Example: Finding the Largest of Three Numbers

Problem: Find the largest among A, B and C.



Algorithm (Step-by-Step)

1. Read three numbers A, B and C.
2. If A is greater than both B and C, set Largest = A.
3. Otherwise, if B is greater than C, set Largest = B.
4. Otherwise, set Largest = C.
5. Display Largest.

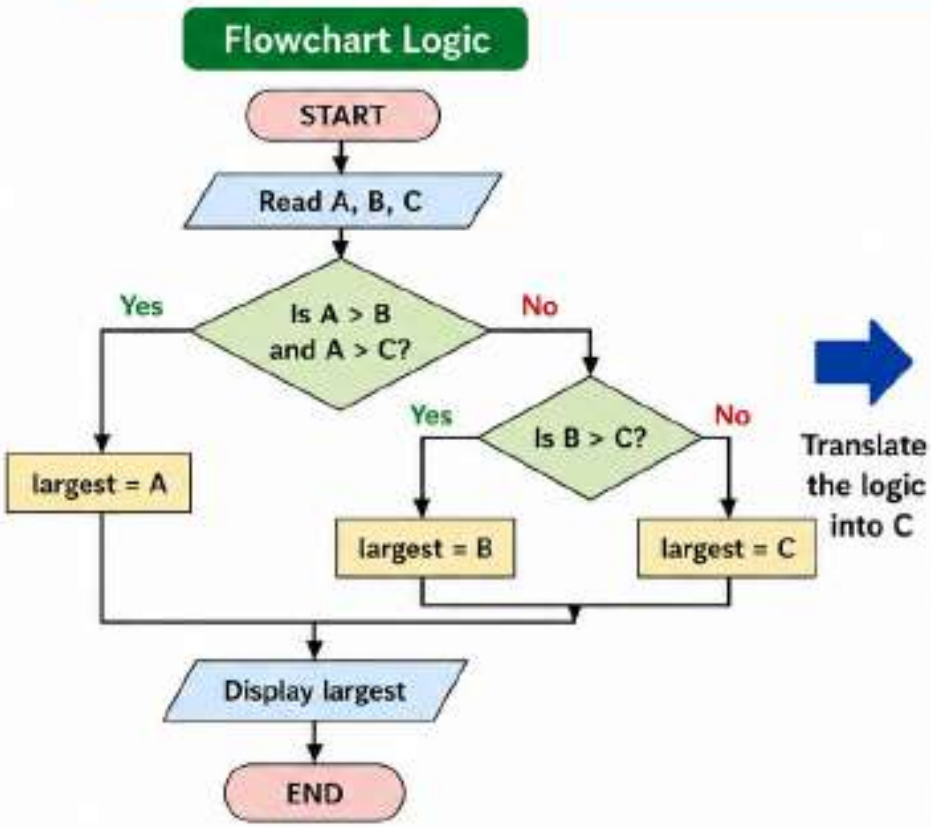
Legend (Flowchart Symbols)

Oval	: Start / End	
Parallelogram	: Input / Output	
Rectangle	: Process / Action	
Diamond	: Decision / Condition	
Arrow	: Flow Direction	

Note: The flowchart represents the logic. The same logic will be written in C in the next step.

From Flowchart to C Program

The logic stays the same – only the representation changes.



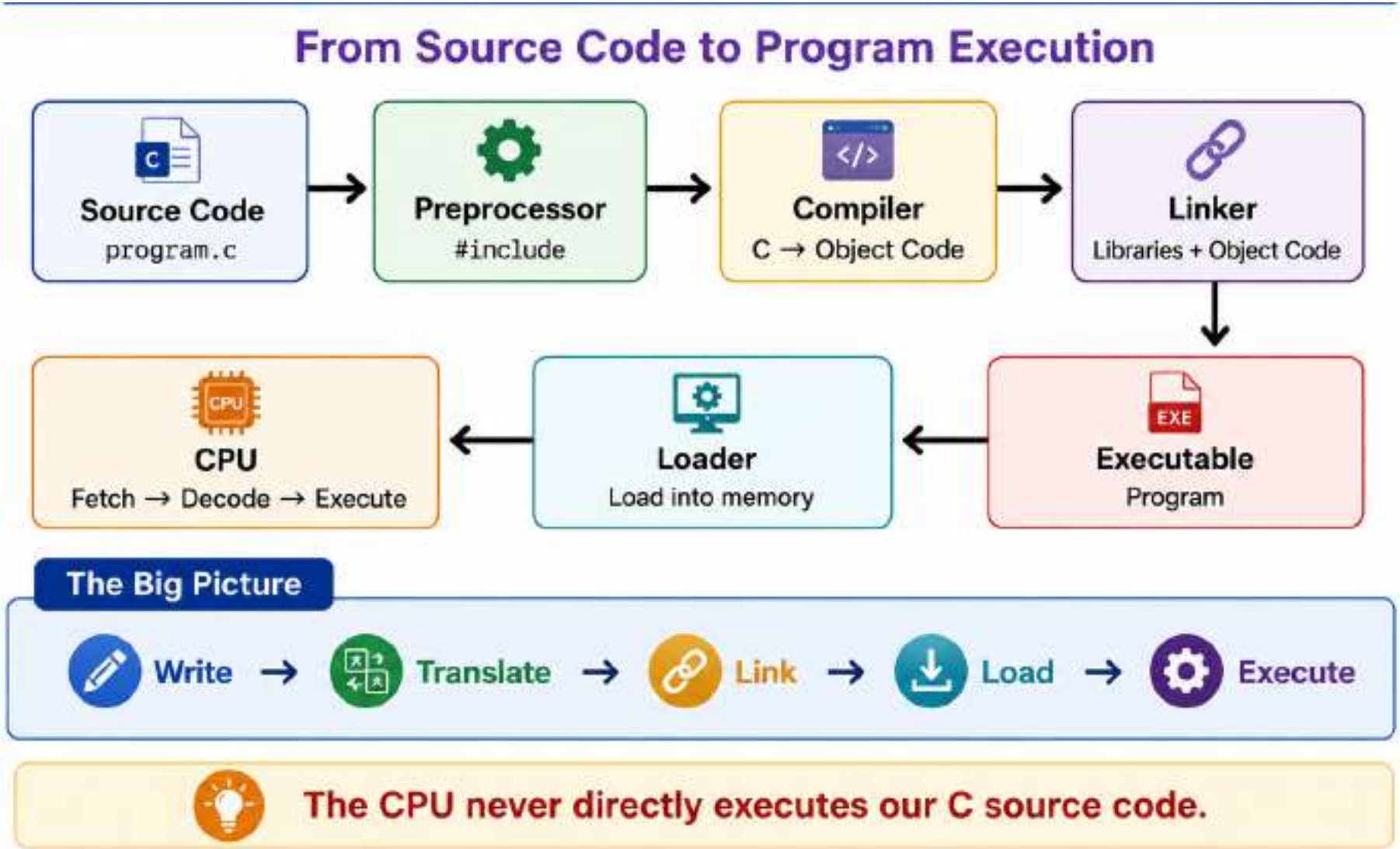
Translate the logic into C

C Program

```c
#include <stdio.h>

int main()
{
    int A, B, C, largest;       // Variable declaration

    scanf("%d %d %d", &A, &B, &C);  // Read input

    if (A > B && A > C)
        largest = A;                // A is largest
    else if (B > C)
        largest = B;                // B is largest
    else
        largest = C;                // C is largest

    printf("Largest = %d", largest);  // Display result

    return 0;
}
```

Problem – What to solve? → **Algorithm** – Step-by-step logic → **Pseudocode** – Structured steps → **Flowchart** – Visual representation → **C Program** – Executable code

What happens When We Run a C Program?



What Will We Learn in Programming for Problem Solving?

From Basic Programming to Real-World Data Processing

UNIT 1
C Fundamentals

- Data Types
- Variables
- Operators
- Expressions
- Decisions
- Boolean Logic

→

UNIT 2
Functions & Pointers

- Problem Decomposition
- Functions
- Arguments
- Pass-by-Value
- Pass-by-Reference
- Pointers

→

UNIT 3
Data Collections

- Arrays
- Strings
- Searching
- Linear Search
- Binary Search
- Sorting

→

UNIT 4
Iteration & Recursion

- Loops
- Break / Continue
- Recursion
- Iteration vs Recursion
- Memory Usage
- Dynamic Memory

→

UNIT 5
Structures & Files

- Structures
- Structure Functions
- Dynamic Memory
- File Handling
- Records
- Text / Line Processing

The Big Journey

Think → Design → Code → Organize Data → Solve Complex Problems

Unit 1 -- C Fundamentals & Decision Making

From Problem Understanding to Basic C Programs

What We Will Learn

- **Data Types and Variables** — `int`, `float`, `char`, constants
- **Operators and Expressions** — arithmetic, relational, logical and assignment operators
- **Assignment and Expressions** — evaluate and trace expressions
- **Decision Making** — `if`, `else if`, `else`
- **Boolean Expressions** — conditions and logical decision making
- **Problem Solving** — convert simple problems into C programs

Guided Practice

Variable tracing → Expressions → Conditions → Decision-making programs

Unit 2 -- Functions, Decomposition & Pointers

From One Large Program to Modular Solutions

What We Will Learn

- **Problem Decomposition** — break a problem into smaller sub-problems
- **Functions** — declaration, definition, calling and return values
- **Function Arguments** — parameters and different ways of passing values
- **Pass-by-Value and Pass-by-Reference** — understand how data is passed to functions
- **Pointers** — addresses, pointer variables and memory access
- **Modular Programming** — design reusable functions for complex problems

Guided Practice

Decompose → Design Functions → Pass Data → Build Modular Programs

Unit 3 -- Arrays, Strings, Searching & Sorting

From Single Values to Collections of Data

What We Will Learn

- **Arrays** — declaration, initialization, indexing and traversal
- **Array Operations** — reading, modifying and processing elements
- **Strings** — character arrays and string manipulation
- **String Functions** — `strlen()`, `strcpy()`, `strcat()`, `strcmp()`, `strchr()`, `strstr()`
- **Searching** — Linear Search and Binary Search
- **Sorting** — Bubble, Selection, Insertion, Quick and Merge Sort

Guided Practice

Store → Traverse → Search → Sort → Process Data

Unit 4 -- Iteration, Recursion & Dynamic Memory

Repeating Solutions and Managing Memory

What We Will Learn

- **Loops** — `for, while and do-while`
- **Iteration** — solve repeated problems systematically
- **Break and Continue** — control loop execution
- **Recursion** — recursive functions and base cases
- **Recursion vs Iteration** — compare solutions and memory behaviour
- **Dynamic Memory Allocation** — `malloc(), calloc(), realloc() and free()`

Guided Practice

Unit 5 -- Structures, Dynamic Data & File Handling

From Simple Data to Real-World Records

What We Will Learn

- **Structures** — group different types of data into a single record
- **Structure Variables** — create, access and modify structured data
- **Functions with Structures** — pass structures to functions
- **Dynamic Memory with Structures** — manage unpredictable or large inputs
- **File Handling** — create, open, read, write and close files
- **File Processing** — process records, text, characters, words and lines

Guided Practice